

Full Stack Development with MERN

Project Documentation

1. Introduction

- **Project Title:** BOOK STORE
- **Team Members:** Venuthurumilli Thanuja

2. Project Overview

• **Purpose :** To develop a full-stack online bookstore where users can browse, search, purchase, and review books, while administrators can manage books, users, orders, and reviews .

• **Features :**

- User Registration & Login
- JWT Authentication
- Browse Books
- Search & Filter Books
- Book Details
- Shopping Cart
- Checkout
- Order Management
- Reviews & Ratings
- Admin Dashboard
- Responsive UI

3. Architecture

- **Frontend** - React.js, Bootstrap, Axios, React Router

The frontend is developed using **React.js** with a component-based architecture. It provides a responsive user interface for user registration, login, book browsing, search and filtering, shopping cart, checkout, order management, reviews, and the admin dashboard. **React Router** is used for navigation, **Axios** for API communication, and **Bootstrap** for responsive design.

- **Backend** - Node.js, Express.js, REST APIs

The backend is built using **Node.js** and **Express.js**, following a RESTful API architecture. It handles user authentication, authorization, book management, order processing, reviews, and admin operations. The backend is organized into **routes, controllers, models, middleware, and configuration files** to ensure scalability and maintainability.

- **Database** - MongoDB using Mongoose

The application uses **MongoDB** as the NoSQL database with **Mongoose** for data modeling. It stores information in collections such as **Users, Books, Orders, Reviews, and Cart**. The backend performs CRUD operations to manage application data efficiently and securely.

4. Setup Instructions

- **Prerequisites:** Node.js

MongoDB

Git

VS Code

- **Installation:** git clone <repository-url>

```
cd client
```

```
npm install
```

```
cd ../server
```

```
npm install
```

Create .env

```
PORT=5000
```

```
MONGO_URI=mongodb://127.0.0.1:27017/BookStoreDB
```

```
JWT_SECRET=change_this_secret_for_production
```

5. Folder Structure

Client (React Frontend)

The client folder contains the React.js application responsible for the user interface. It includes reusable components, pages, routing, styles, API integration using Axios, and static assets. The frontend provides features such as user authentication, book browsing, search and filtering, shopping cart, checkout, order management, reviews, and the admin dashboard.

```
frontend/
| — public/
| — src/
|   | — components/
|   | — pages/
|   | — context/
|   | — services/
|   | — assets/
|   | — App.js
|   | — index.js
| — package.json
```

Server (Node.js Backend)

Server folders contain Node.js and Express.js backend. It manages REST APIs, authentication, business logic, database operations, and application security. The backend is organized using controllers, models, routes, middleware, and configuration files to ensure a scalable and maintainable architecture.

```
backend/
| — config/
| — controllers/
| — middleware/
| — models/
| — routes/
| — utils/
| — server.js
| — package.json
```

6. Running the Application

- Provide commands to start the frontend and backend servers locally.
 - **Frontend:** cd frontend
npm start
 - **Backend:** cd backend
npm start

7. API Documentation

Method	Endpoint	Description
POST	/api/users/register	Register User
POST	/api/users/login	User Login
GET	/api/books	View Books
GET	/api/books/:id	Book Details
POST	/api/cart	Add to Cart
POST	/api/orders	Place Order
GET	/api/orders	View Orders
POST	/api/reviews	Add Review
PUT	/api/books/:id	Update Book
DELETE	/api/books/:id	Delete Book
GET	/api/users/profile	User Profile

8. Authentication

The BookStore application uses **JWT (JSON Web Token)** for secure user authentication and authorization. During login, users provide their email and password. The password is verified using **bcrypt**, which securely stores encrypted passwords in the database.

After successful authentication, the server generates a **JWT token** that is sent to the client. This token is included in subsequent API requests to access protected resources. Middleware validates the token before allowing access to secured routes.

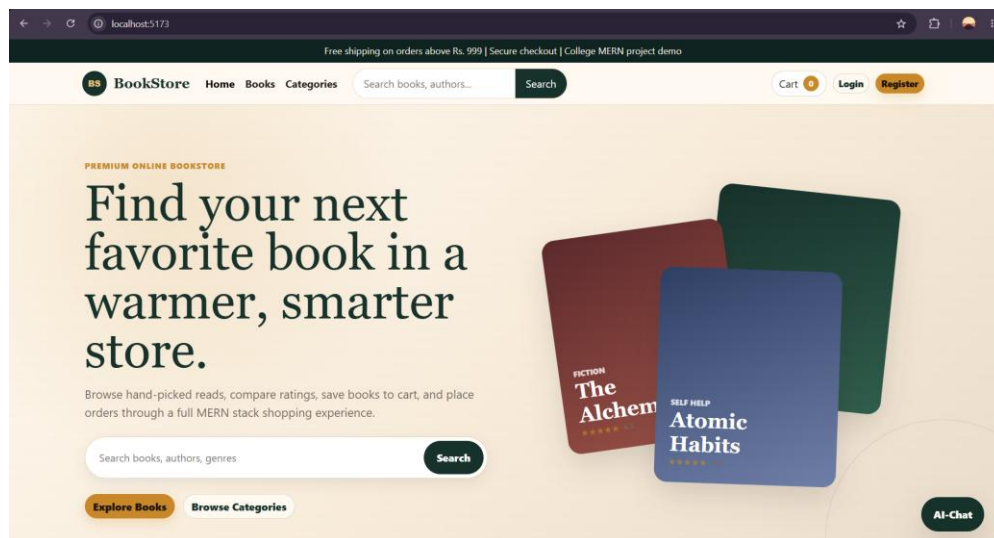
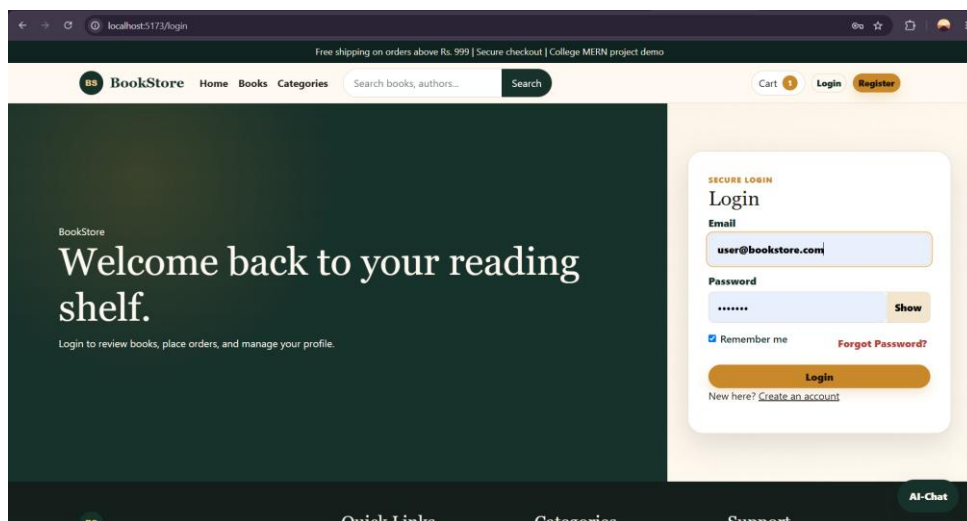
The application implements **Role-Based Authorization**, where users are assigned either **User** or **Admin** roles. Regular users can browse books, manage their cart, place orders, and submit reviews, while administrators have additional privileges to manage books, users, orders, and reviews.

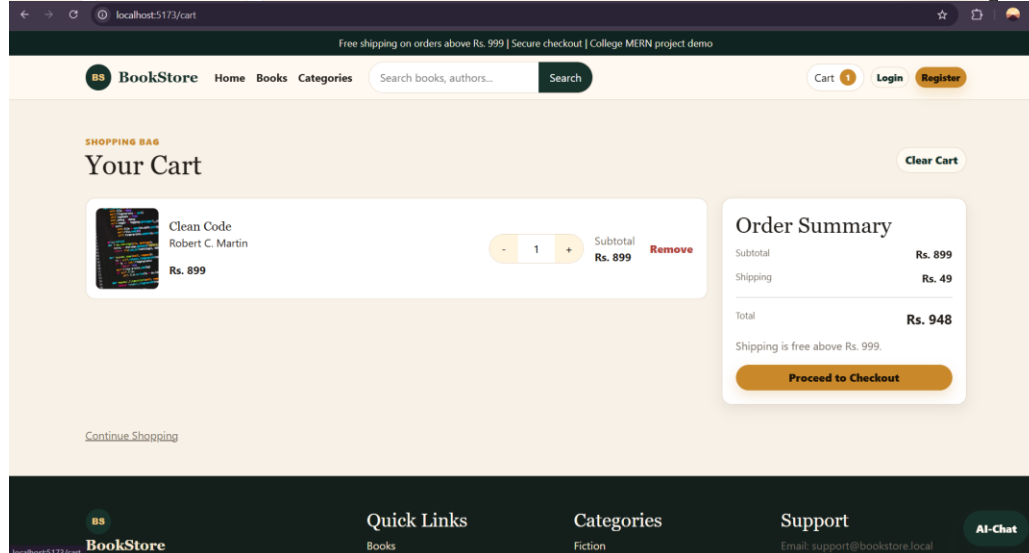
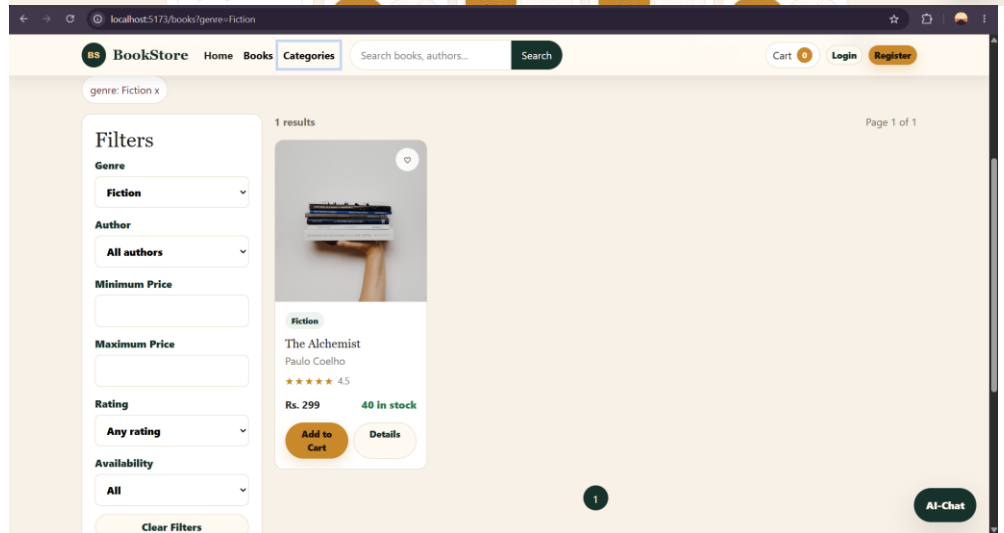
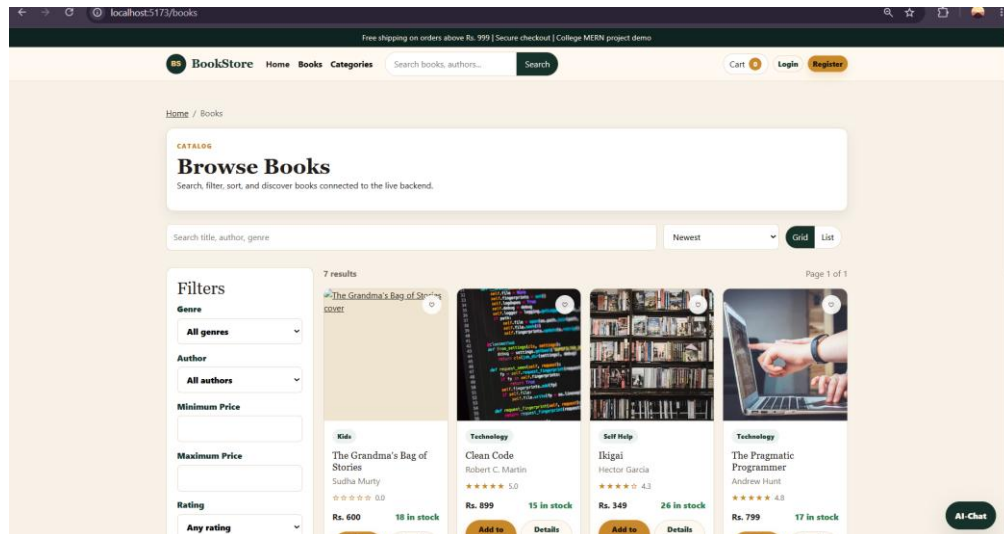
Protected Routes ensure that only authenticated users can access restricted pages such as the user profile, cart, checkout, and order history. Admin routes are accessible only to users with administrative privileges.

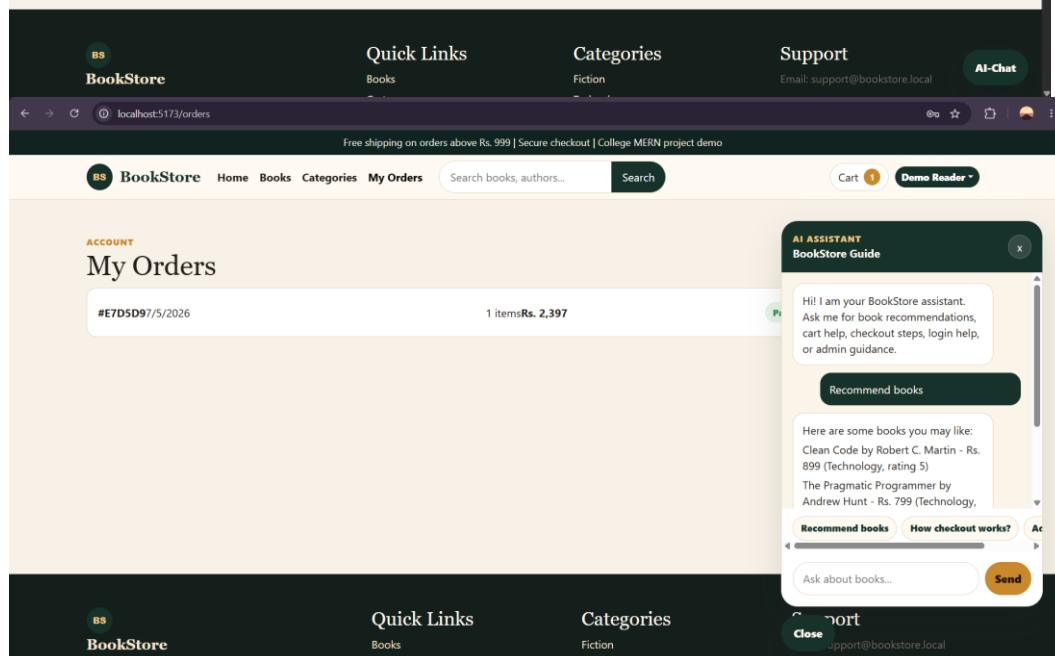
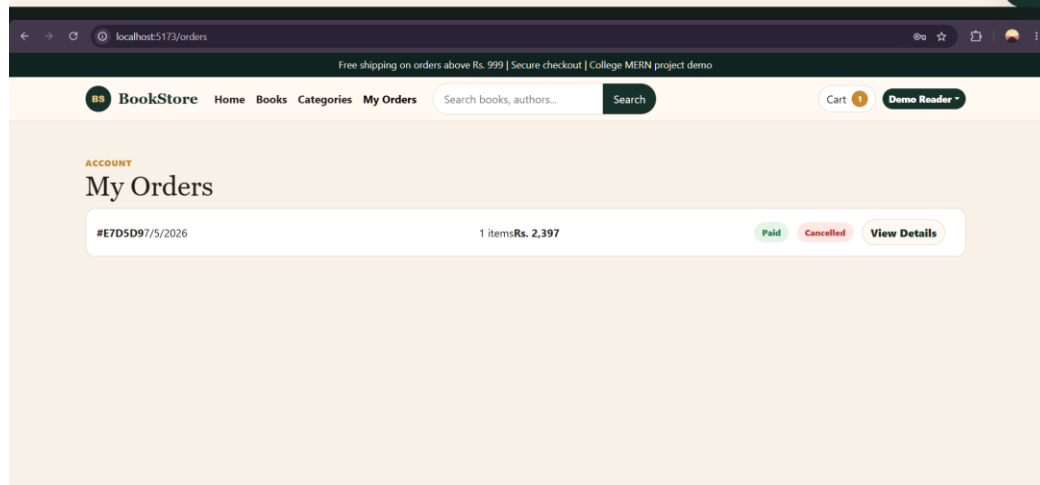
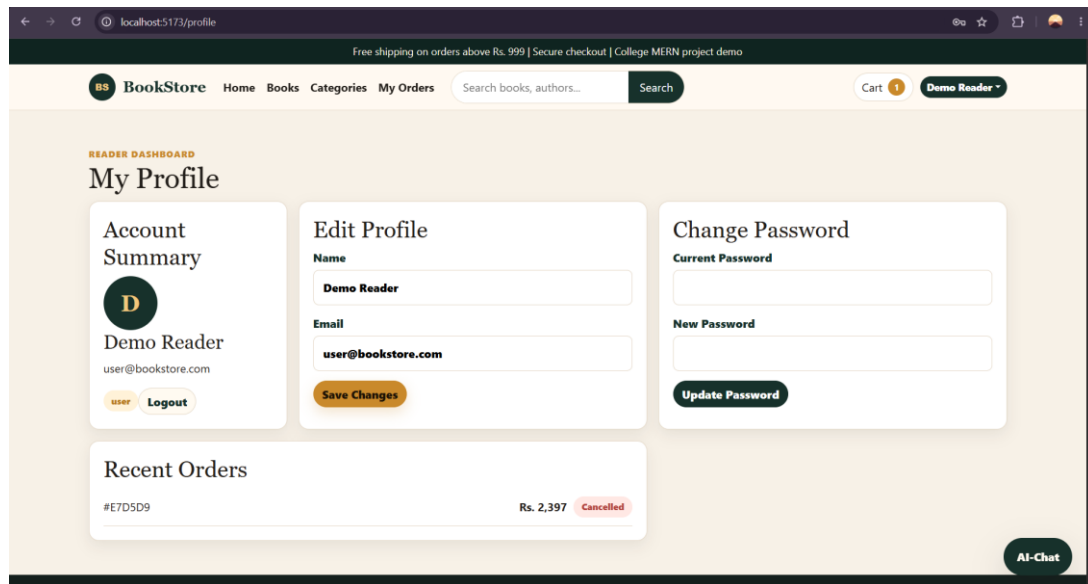
Authentication Methods Used

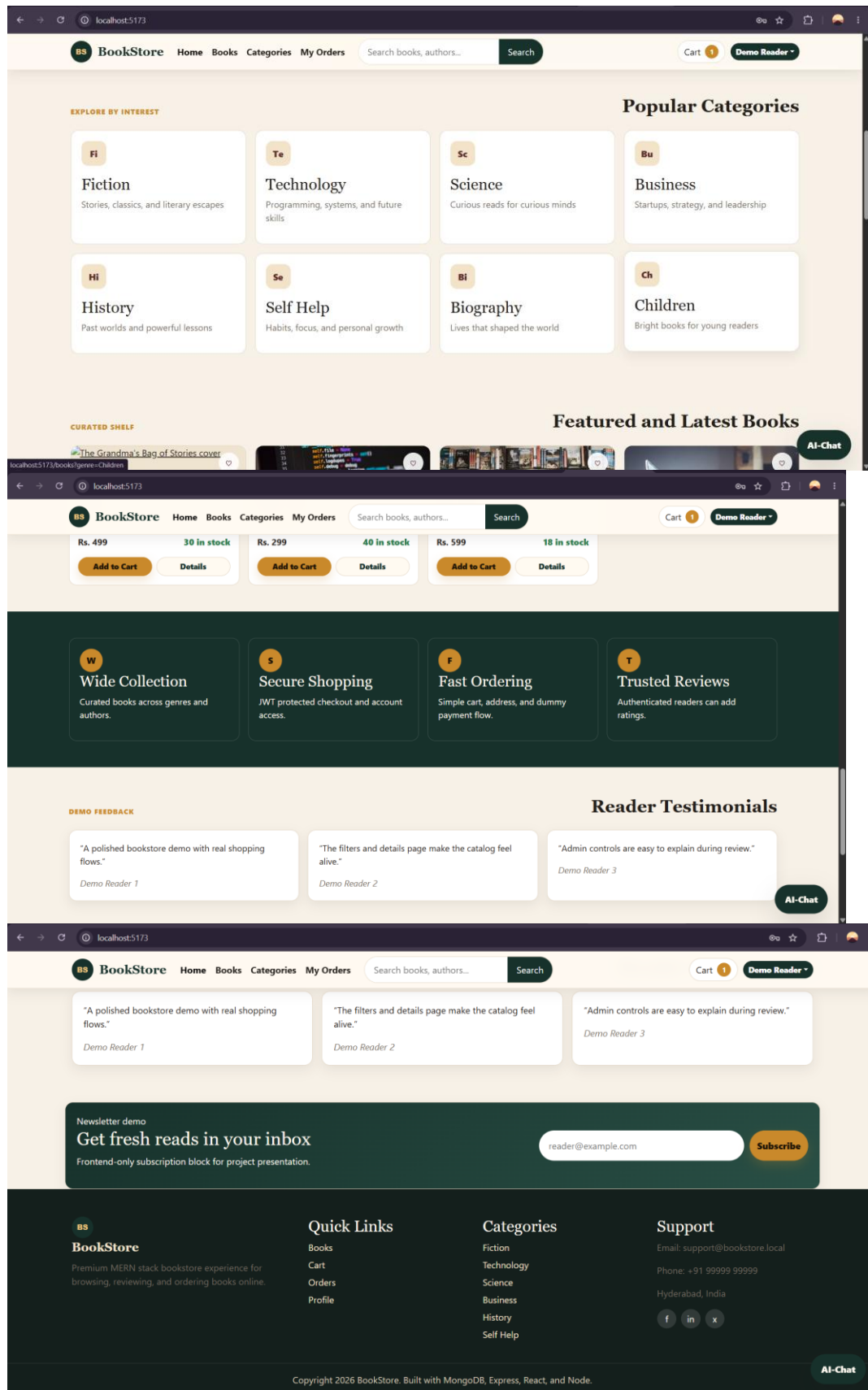
- **JWT (JSON Web Token)** – Secure authentication using access tokens.
- **bcrypt** – Password encryption and secure password verification.
- **Role-Based Authorization** – Separate access for User and Admin.
- **Protected Routes** – Restricts access to authenticated and authorized users only.

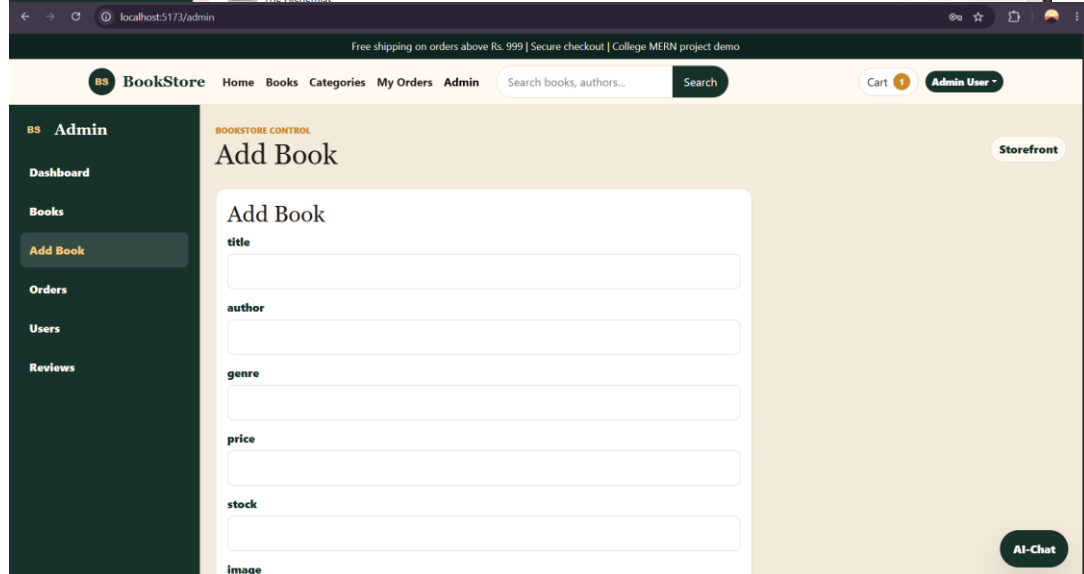
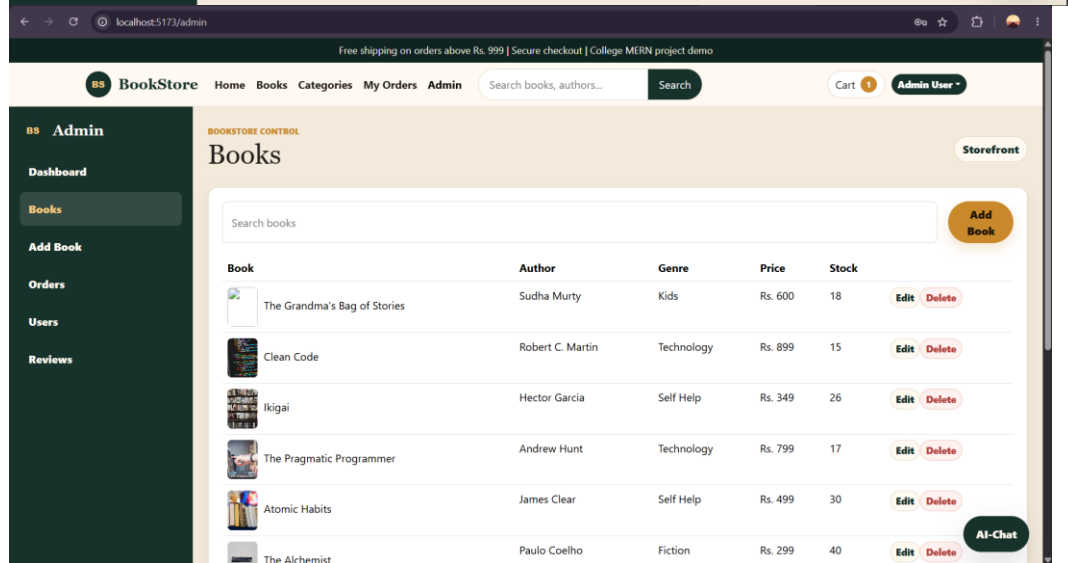
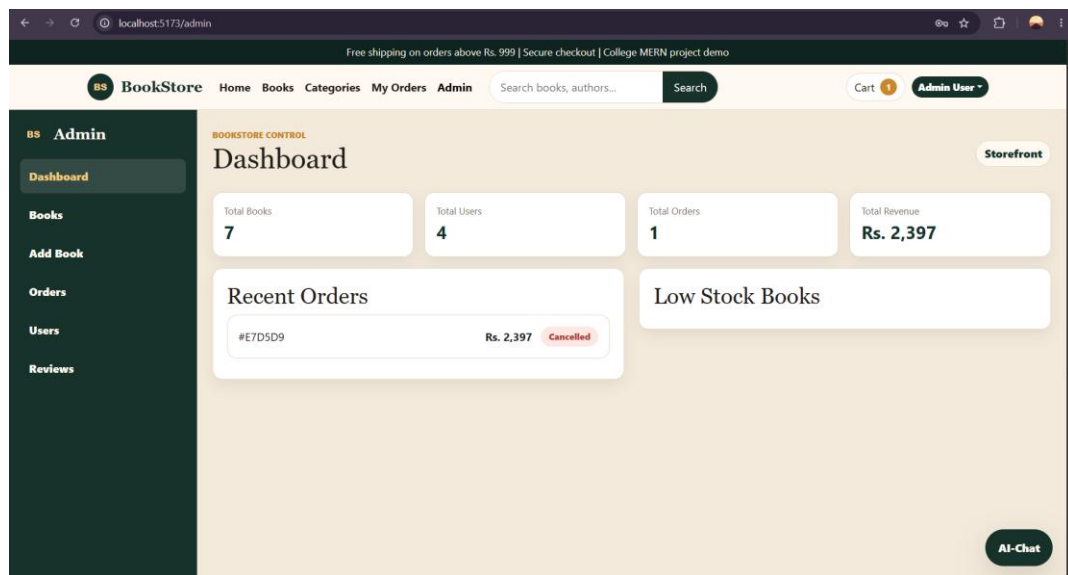
9. User Interface

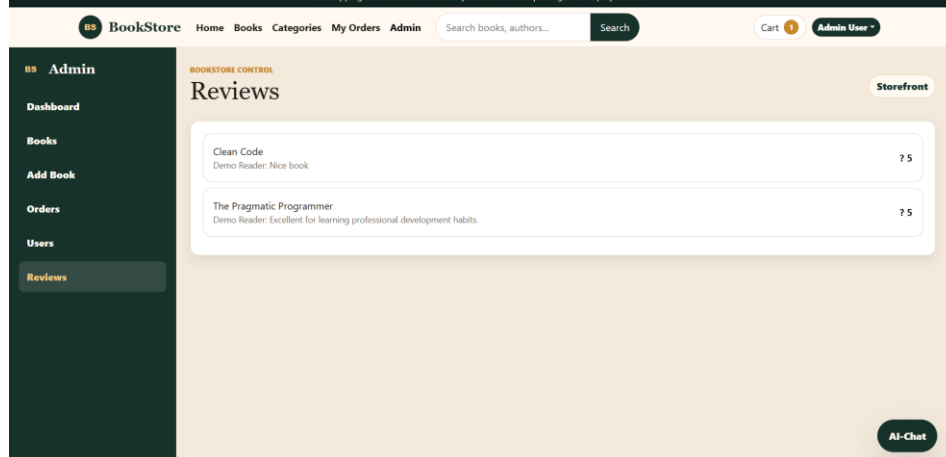
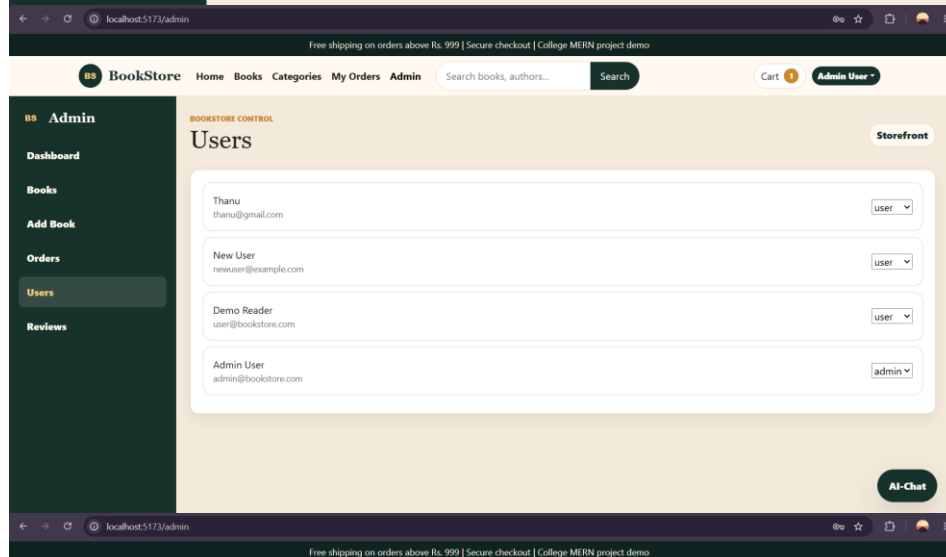
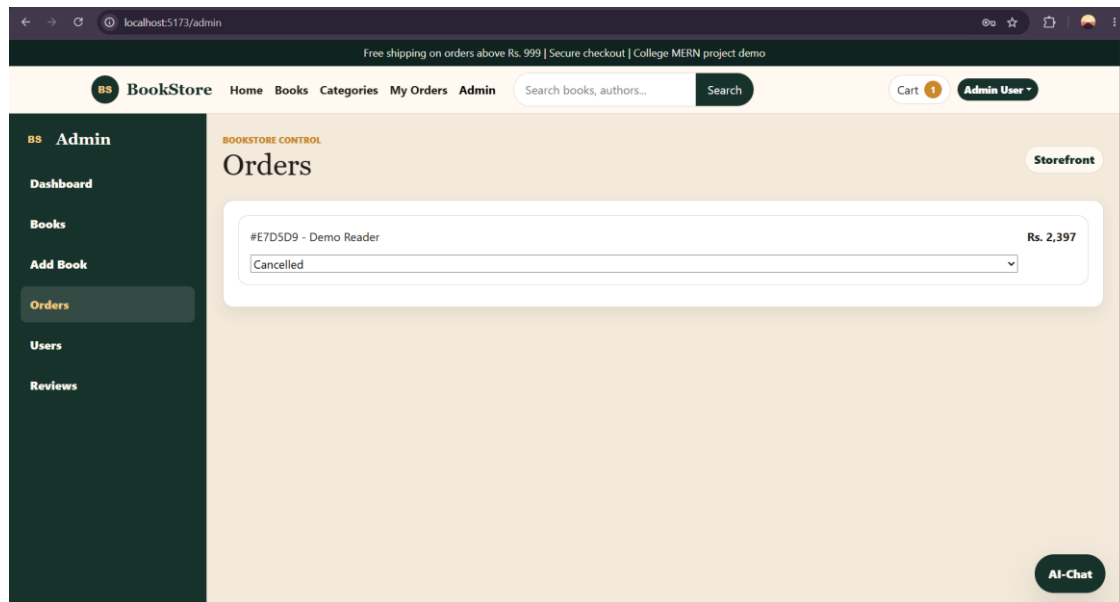












10. Testing

Tools Used

- Postman
- MongoDB Compass
- Chrome Browser

Testing Performed

- Registration
- Login
- Search
- Cart
- Checkout
- Orders
- CRUD Operations
- Admin Functions

11. Screenshots or Demo

BookStore Project Output Demo Video Link:

https://drive.google.com/drive/folders/13aU6ki1u1hbIdqX76fi4Id9-ShWxj73D?usp=drive_link

12. Known Issues

- Payment Gateway not integrated.
- Email notifications not implemented.
- Wishlist feature not available.
- AI Recommendations planned for future.

13. Future Enhancements

- Online Payment Gateway
- Wishlist
- AI Book Recommendation
- Email Notifications
- Chatbot Support
- Multi-language Support
- Inventory Analytics
- Mobile Application